Entrupy Engineering — Intern Assignment

Submission: Public GitHub repository

Deadline: 48 hours from when you receive this document

The Problem

E-commerce businesses need to track competitor pricing across platforms. When prices change, downstream systems (alerts, analytics) need to know — sometimes immediately, sometimes in batch.

Your task: build a **Product Price Monitoring System** that collects product data from marketplaces, tracks changes in real time.

What Your System Must Do

1. Collect

You will be provided with sample product data across three marketplaces, assess and study the provided data and design accordingly, the sample data might have a lot of fields, make sure the relevant ones are included in your architecture, these are the marketplaces:

- Grailed
- Fashionphile
- 1stdibs

2. Store

Design a database (SQLite or PostgreSQL) that supports:

- Current product state
- Price history over time
- Querying by category, source, and price range

Think about what happens when the same product appears in multiple sources. Think about what happens when your price history table has millions of rows.

3. Serve

Build an API that lets consumers:

- Trigger a data refresh
- Browse and filter products
- View a single product's details and price history
- See aggregate analytics (totals by source, averages by category, etc.)

Your API should authenticate consumers, track their usage per request, and handle bad input gracefully.

4. Notify

When a data refresh detects a price change, your system should be able to notify interested parties. How you implement this is up to you — webhooks, polling, event log, or something else. Whatever you choose, make it reliable: handle delivery failures, don't lose events, and don't block the fetch process.

5. Display

Build a web interface (vanilla JS, React, Vue, or server-rendered templates) that shows:

- A dashboard with aggregate stats
- A browsable, filterable product list
- Product detail with price history

It doesn't need to be pretty. It needs to be functional, readable, and responsive.

What We're Evaluating

Area	Weight
Does it work? Endpoints respond correctly, data flows end-to-end, UI renders real data	25%
Code quality. Readable, organized, typed, no dead code, errors handled properly	25%
System design. Schema choices, API structure, event architecture, how you handle scale concerns	20%
Testing. Meaningful tests (not just happy path), edge cases, at least 8 tests	15%
Documentation. Clear README with setup instructions, design rationale, and known limitations	10%
Git workflow. Incremental commits	5%

Your README

This is as important as the code. Include:

1. **How to run it** — exact commands, no guesswork
2. **API documentation** — what endpoints exist, how to call them, example responses
3. **Design decisions** — address at least these:
    - How does your price history scale? What happens at millions of rows?
    - How did you implement notification of price changes, and why that approach over alternatives?
    - How would you extend this system to 100+ data sources?
4. **Known limitations** — what would you improve with more time?

Technical Constraints

- Python 3.9+
- SQLite or PostgreSQL
- Async data fetching with retry logic

- `requirements.txt` with pinned versions
- `.gitignore` that excludes the obvious

Everything else — project structure, libraries, patterns, architecture — is your call. We care about *why* you chose what you chose.

Notes

- **Code originality:** We review your git history. A single massive commit is a red flag. Develop incrementally.
- **Assumptions:** If something is ambiguous, make a reasonable choice and document it. That's part of the evaluation.